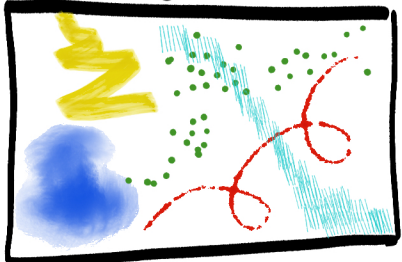


MAKE A DRAWING TOOL

ps.js



openprocessing.org/amy-bw/2891452

Scan here

This zine goes with this tutorial!

```

1 let currentBrush = 1;
2 function setup() {
3   createCanvas(400, 400);
4   background(220);
5 }
6
7 function draw() {
8   if (keyPressed) {
9     if (currentBrush == 1) {
10      bubbleBrush();
11    }
12    if (currentBrush == 2) {
13      confettiBrush();
14    }
15  }
16 }
17
18 function bubbleBrush() {
19   if (mouseIsPressed) {
20     noStroke();
21     fill(random(255), random(255), random(255));
22     rect(mouseX, mouseY, 10, 10);
23   }
24 }
25
26 function confettiBrush() {
27   if (mouseIsPressed) {
28     noStroke();
29     fill(120, 180, 255, 80);
30     ellipse(mouseX, mouseY, 25, 25);
31   }
32 }

```

Don't forget to add this variable to which key was pressed.

4 KeyPressed()

function keyPressed() {

if (key == "1") {

currentBrush = 1;

if (key == "2") {

currentBrush = 2;

}

}

function draw() {

if (currentBrush == 1) {

bubbleBrush();

if (currentBrush == 2) {

confettiBrush();

}

}

}

Then use draw() to check which brush was chosen.

5

Key holds which key you pressed. "1" goes in quotes because a key is a character, not a number.

When you press a key, keyPressed runs

more as needed

Add

6

SWITCH BETWEEN BRUSHES

1 Make your brush into a function:

```

function confettiBrush() {
  if (mouseIsPressed) {
    noStroke();
    fill(random(255), random(255), random(255));
    ellipse(mouseX, mouseY, 10, 10);
  }
}

```

2 Great! Now, "call" the function in draw

```

function draw() {
  confettiBrush();
}

```

3 Make at least two different brushes into functions.

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(220);
}

function draw() {
  confettiBrush();
  bubbleBrush();
}

function confettiBrush() {
  if (mouseIsPressed) {
    noStroke();
    fill(random(255), random(255), random(255));
    ellipse(mouseX, mouseY, 10, 10);
  }
}

function bubbleBrush() {
  if (mouseIsPressed) {
    noStroke();
    fill(random(255), random(255), random(255));
    rect(mouseX, mouseY, 10, 10);
  }
}

```

```

function setup() {
  createCanvas(400, 400);
  background(220);
}

function draw() {
  if (mouseIsPressed) {
    fill(255, 0, 150);
    ellipse(mouseX, mouseY, 20, 20);
  }
}

```

createCanvas(400, 400)

width height

DID YOU KNOW?

You can use windowWidth, windowHeight to make the canvas auto-adjust

Your turn: Change some numbers. Press to run the code again. What do you notice?

COLORS

These functions take a color:

background() ← paints whole canvas

fill() ← paints inside shapes

stroke() ← paints the outline

GRAYS SCALE: one number

fill(0) ← black

fill(128) ← gray

fill(255) ← white

RGB: three numbers

fill(255, 0, 0) ← red

fill(0, 255, 0) ← green

fill(255, 200, 0) ← yellow-orange

red amount green amount blue amount

NOTE: Numbers range from 0-255. You're mixing light, not paint, so 255, 255, 255 gives white

See-through Colors (alpha)

fill(0, 100) ← see-through black

fill(255, 0, 0, 50) ← ghostly red

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(220);
}

function draw() {
  if (mouseIsPressed) {
    noStroke();
    fill(random(255), random(255), random(255));
    rect(mouseX, mouseY, 10, 10);
  }
}

```

LOOKS LIKE CONFETTI!

MEET random()

Each time it runs, random(255) gives back a surprise number. This ↑ one gives back a number from 0 up to ~255

Pick a range: min max implied zero min

fill(random(200, 255), 0, random(100));

Red value randomly chosen between 200-255

Green value always 0

Blue value randomly chosen between 0-100

SHAPES

rect(mouseX, mouseY, 10, 20)

These can take static x- or y-coordinates or use mouseX mouseY to draw the shape where the cursor is.

rect(x, y, w, h) ← in other words: width of the rectangle height of the rectangle

ellipse(x, y, w, h)

line(x1, y1, x2, y2)

triangle(x1, y1, x2, y2, x3, y3)

quad(x1, y1, x2, y2, x3, y3, x4, y4)

You can use random() for any value